

# Multimodal GenAI: Text, Image & Code

Diffusion · Stable Diffusion · DALL·E · Codex and Beyond

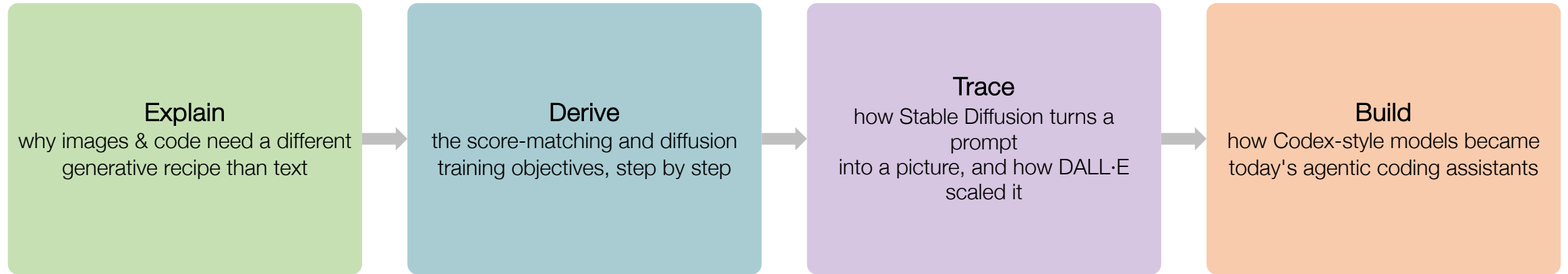
**Dr. Mainack Mondal**

CSE, IIT Kharagpur

B3B Advanced AI Course – Cohort 3

30 September 2026

# By the End of This Session, You Will Be Able To...



We will return to this roadmap before every module — look for the module name in red.

# Today's Roadmap

- 01 Recap: From Language to Pixels
- 02 Why Generating Images Is Hard
- 03 Score-Based Generative Models
- 04 Denoising Diffusion Models
- 05 Guidance: Classifier & Classifier-Free
- 06 Latent Diffusion & Stable Diffusion
- 07 Cutting-Edge Text-to-Image: DALL·E to Today
- 08 Code Generation: Codex to Agentic Coding
- 09 Wrap-up & Takeaways

# Recap: From Language to Pixels

## 01 Recap: From Language to Pixels

02 Why Generating Images Is Hard

03 Score-Based Generative Models

04 Denoising Diffusion Models

05 Guidance: Classifier & Classifier-Free

06 Latent Diffusion & Stable Diffusion

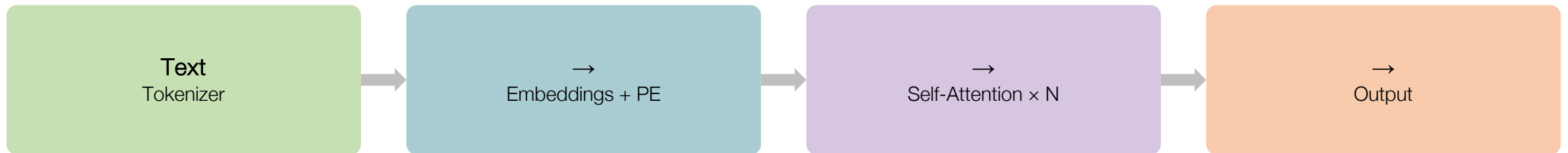
07 Cutting-Edge Text-to-Image: DALL·E to Today

08 Code Generation: Codex to Agentic Coding

09 Wrap-up & Takeaways

# Last Time: Text as a Prediction Problem

- A decoder-only Transformer reads tokens  $x_0 \dots x_{i-1}$  and outputs a distribution over the next token:  $\Pr(. \mid x_0, \dots, x_{i-1})$ .
- Pipeline: Tokenizer  $\rightarrow$  Embedding + Positional Encoding  $\rightarrow$  Self-Attention ( $\times N$  layers)  $\rightarrow$  softmax over the vocabulary.
- Generation = sample one token at a time from this distribution, feed it back in, repeat — autoregression.
- Today's question:
  - can this same “model a distribution, then sample from it” idea generate images and code — not just text?



# “Autoregressive” — A Word You'll Hear All Day

- **Autoregressive** just means: predict the next thing using everything generated so far, then feed that prediction back in as new input, and repeat.

1

Start with a prompt

"The cat sat on the"

2

Predict one next token

model looks at the whole sentence so far → predicts "mat"

3

Feed the prediction back in

new input is now "The cat sat on the mat" — including the word we just generated

4

Repeat

predict the next token again, using the now-longer sentence — over and over

“Auto” (self) + “regressive” (regress onto its own earlier outputs) — the model's own past predictions become part of its next input.

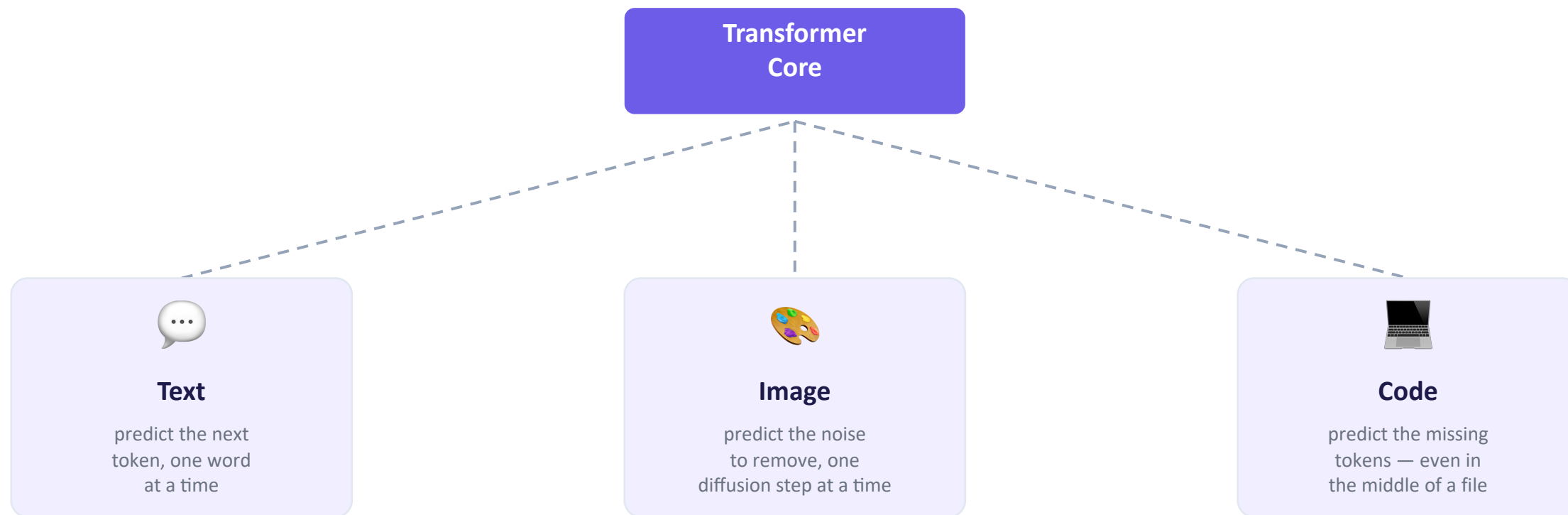
# One Recipe, Different Challenges for Three Modalities

- Every generative model, whatever it produces, needs two abilities:
  - **Probability evaluation** — score how likely a candidate output is
  - **Sampling** — draw new, plausible outputs from that distribution

| Modality | “Vocabulary”                         | Why it's different                                                                     |
|----------|--------------------------------------|----------------------------------------------------------------------------------------|
| Text     | tens of thousands of discrete tokens | next-token prediction works beautifully                                                |
| Image    | a grid of continuous pixel values    | far higher-dimensional and continuous                                                  |
| Code     | discrete tokens like text            | long-range structure (imports, signatures, repo context) rewards a different objective |

# One Engine, Three Different Steering Wheels

- Same underlying Transformer weights and attention math stays
  - what changes is what we predict, and how we turn predictions into an output.



✨ **INSIGHT** One engine, but three steering wheels — the weights and attention math never change; only the training objective and the output head do.

# Why Generating Images Is Hard

- 01 Recap: From Language to Pixels
- 02 **Why Generating Images Is Hard**
- 03 Score-Based Generative Models
- 04 Denoising Diffusion Models
- 05 Guidance: Classifier & Classifier-Free
- 06 Latent Diffusion & Stable Diffusion
- 07 Cutting-Edge Text-to-Image: DALL·E to Today
- 08 Code Generation: Codex to Agentic Coding
- 09 Wrap-up & Takeaways

# The Data Distribution Is Extremely Complex

- Natural images are points sampled from an unknown, high-dimensional distribution  $P_{\text{data}}(\mathbf{x})$ 
  - $P_{\text{data}}(\mathbf{x})$  : every possible photo of a dog lives somewhere on this landscape.
- A generative model builds its own model distribution  $P_{\theta}(\mathbf{x})$  and tries to make it match  $P_{\text{data}}(\mathbf{x})$  as closely as possible.
- A good model must do two things well: evaluate how probable a candidate image is, and sample brand-new, never-seen images from high-probability regions.

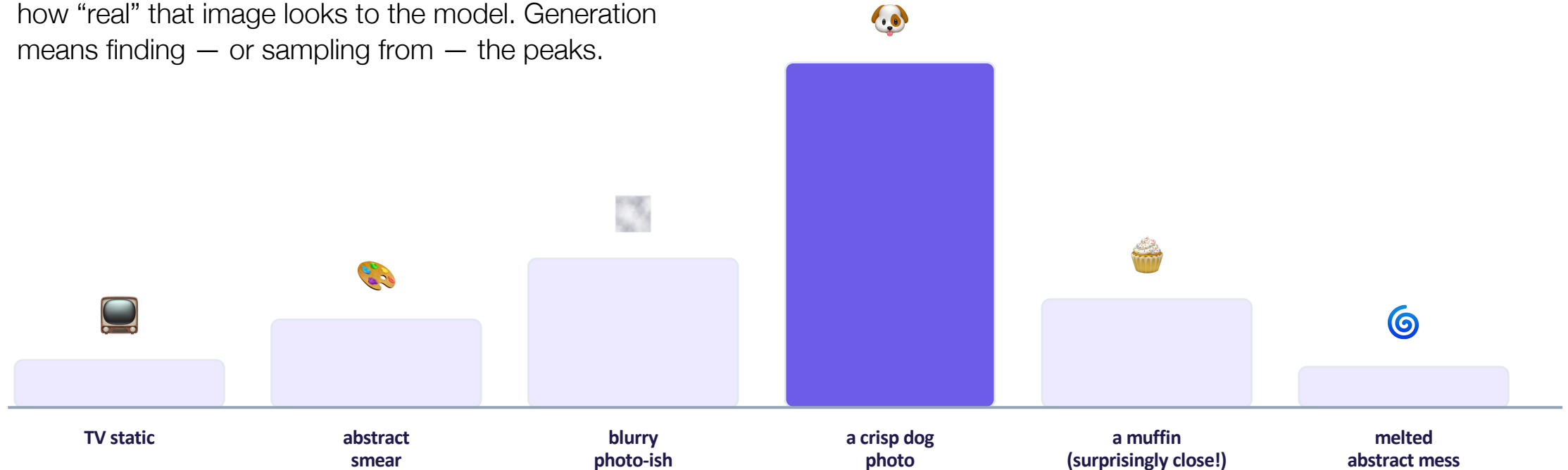
Classic illustration: a chihuahua face scores high probability under  $P(\text{dog})$ ; a blueberry muffin scores low — yet superficially both are “round, spotty, brown” images.

High probability → looks like a photo

Low probability → looks like noise / a muffin mistaken for a chihuahua

# A Landscape of Probability

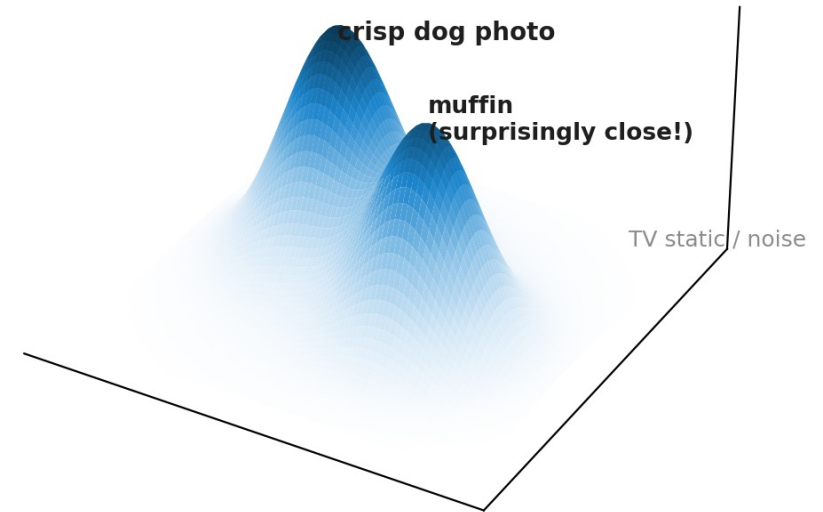
- Imagine every possible image laid out side by side. Height = how “real” that image looks to the model. Generation means finding — or sampling from — the peaks.



✨ **INSIGHT** The chihuahua-vs-muffin problem: two very different “things” sit at similarly high probability if they share round, speckled, warm-toned structure — probability tracks statistical pattern, not meaning.

# A Landscape of Probability

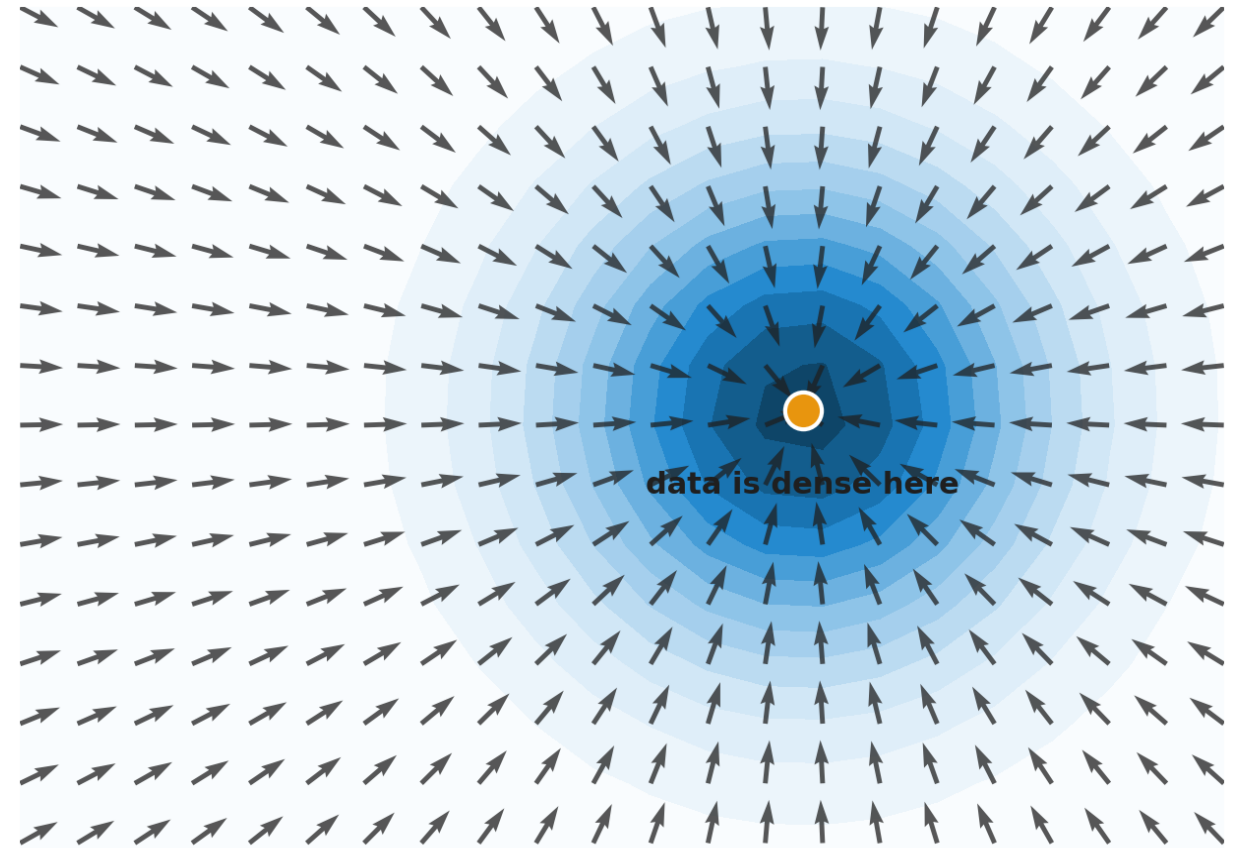
- Imagine every possible image laid out side by side. Height = how “real” that image looks to the model. Generation means finding — or sampling from — the peaks.



✨ **INSIGHT** The chihuahua-vs-muffin problem: two very different “things” sit at similarly high probability if they share round, speckled, warm-toned structure — probability tracks statistical pattern, not meaning.

# The Score Field: Arrows Pointing Home

- Every point in space gets an arrow — its score — telling a sample which way to move to look more like real data.



✨ **INSIGHT** Near the island the arrows are short and confident; far away (low-density “wastelands”) they are unreliable — exactly why the multi-noise-level trick from the previous slide is needed.

## Before We Move On...

Q Why can't we just reuse next-token text prediction directly on raw pixels?

We technically can (pixel-autoregressive models exist), but images have far more “tokens” (millions of pixel values) and no natural left-to-right reading order, so autoregression is slow and doesn't exploit 2D structure well. This motivated a different family: score- and diffusion-based models.

# Score-Based Generative Models

- 01 Recap: From Language to Pixels
- 02 Why Generating Images Is Hard
- 03 Score-Based Generative Models**
- 04 Denoising Diffusion Models
- 05 Guidance: Classifier & Classifier-Free
- 06 Latent Diffusion & Stable Diffusion
- 07 Cutting-Edge Text-to-Image: DALL·E to Today
- 08 Code Generation: Codex to Agentic Coding
- 09 Wrap-up & Takeaways

# Two Words We'll Lean On All Module Long

## Log-probability

$$\log P(x)$$

“Log” just means “take the logarithm.” Why bother? Probabilities are tiny numbers that are awkward to multiply ( $0.001 \times 0.002 \times \dots$ ). Logs turn multiplication into addition and turn “how likely” into a smooth landscape we can climb.

logs turn tiny multiplied probabilities into an addable, smooth landscape

$$P(x) = 0.5 \rightarrow \log P(x) \approx -0.69$$

$$P(x) = 0.01 \rightarrow \log P(x) \approx -4.6$$

More likely  $\rightarrow \log P(x)$  closer to 0 (less negative)

## Gradient

$$\nabla_x f(x)$$

The gradient is just “which direction makes  $f(x)$  increase fastest, and how steeply.” Standing on a foggy hillside, you can't see the peak — but your feet can feel which way is uphill. That felt slope is the gradient.

which direction makes  $f(x)$  increase fastest — the “felt slope” underfoot

$$f(x) = -(x-5)^2/2 \quad (\text{a hill peaking at } x=5)$$

At  $x=2$ , slope = +3  $\rightarrow$  uphill is to the right

At  $x=8$ , slope = -3  $\rightarrow$  uphill is to the left

More likely  $\rightarrow \log P(x)$  closer to 0 (less negative). Standing on a foggy hillside you can't see the peak — but your feet feel which way is uphill. That felt slope is the gradient.

# The Partition Function — Why It's a Problem

- To turn “scores” into real probabilities that sum to 100%, you need a normalizer — the **partition function  $Z_\theta$** .
  - Now think what are properties of the normalizer so that the probabilities sum to 100%

# The Partition Function — Why It's a Problem

- To turn “scores” into real probabilities that sum to 100%, you need a normalizer — the **partition function  $Z_\theta$** .
  - Now think what are properties of the normalizer so that the probabilities sum to 100%
- **The catch:** for a 512×512 image, “every conceivable image” is a number so large it dwarfs the count of atoms in the observable universe. There is no shortcut to sum over all of them.

Analogy: grading every student's essay as a raw “score,” then converting to percentages of the class. Easy with 30 students — impossible if the “class” is every possible essay that could ever be written.

✨ **INSIGHT** This is precisely the wall that stopped classical density models — and exactly what the score function (next slide) lets us walk straight through.

# Solution: From Density Estimation to Score Matching

Hyvärinen, JMLR 2005

Song & Ermon, NeurIPS 2019

- Classical approach:
  - model  $P_{\theta}(x) = q(x;\theta) / Z_{\theta}$ , where  $Z_{\theta}$  is a normalizing constant
  - intractable to integrate over all of image space.
- Hyvärinen (2005):
  - Sidestep  $Z_{\theta}$  entirely by matching **the score function** — the gradient of log-probability — instead of the probability itself.
- Song & Ermon (2019):
  - Scale score matching up to deep networks and natural images (“NCSN”).

# The Score Function

- Analogy: hiking uphill in dense fog. You can't see the summit (needs the impossible  $Z_\theta$ ), but you can always feel which way the ground slopes — and step that way. The score is exactly that felt slope.
- The score is the gradient of log-probability w.r.t.  $x$  — “which way is uphill, from here?”

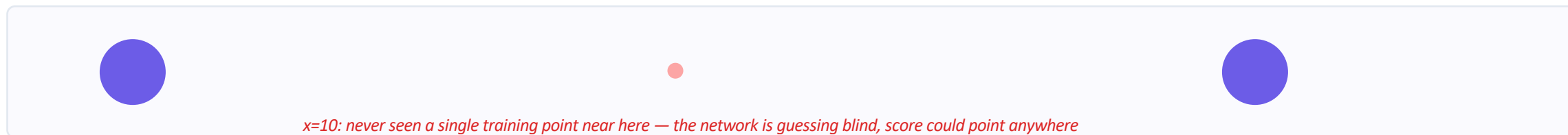
$$s_\theta(x) = \nabla_x \log P_\theta(x) = -\nabla_x f_\theta(x) \quad (\text{the } \log Z_\theta \text{ term vanishes})$$

✨ **INSIGHT** Crucially,  $Z_\theta$  disappears when we take the gradient, since  $\log Z_\theta$  is constant w.r.t.  $x$ , so  $\log Z_\theta = 0$ . We never need to compute it. Training just adjusts  $s_\theta$  until its felt slope matches the real data's slope everywhere.

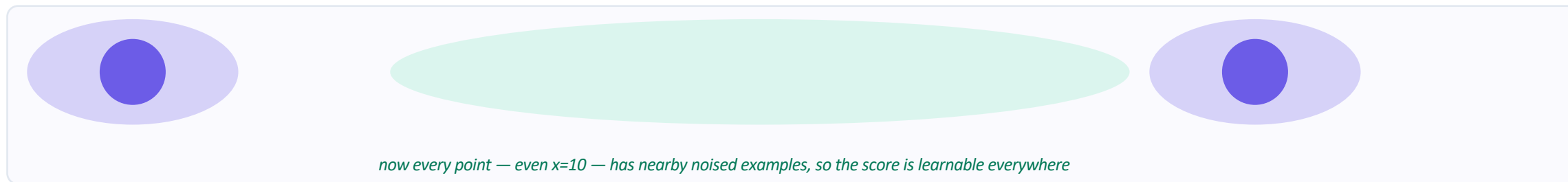
# Scores Are Only Accurate Where Data Is Dense

- Concrete example: our “data” is two tiny islands — real points near  $x=2$  and  $x=20$ 
  - total emptiness at  $x=10$ , where the network is guessing blind.

**Without noise: two isolated islands**



**Add Gaussian noise (bell-curve-shaped random static): islands spread and overlap**



- The fix: corrupt the data with several strengths of Gaussian noise  $\sigma_1 < \sigma_2 < \dots < \sigma_n$ . **More noise = more overlap = score learnable further from real data** — but also less detail preserved.
- So we train one noise-conditional network  $s_\theta(x, \sigma)$ : same network, but “how noisy is this point right now” is an extra input, like a dial it can read.

# A Toy Score example

Simplify to 1 dimension. Suppose our “data” is a single cluster of points centred at  $x = 5$ , and our current noisy sample sits at  $x = 2$ .

A simple bell-curve (Gaussian) density peaking at 5 has log-density  $\log P(x) = -(x-5)^2/2 + \text{const.}$

The score is its derivative:  $\nabla_x \log P(x) = -(x-5)$ .

$$x = 2$$

score =  $-(2-5) = +3$   
push right, toward 5



$$x = 8$$

score =  $-(8-5) = -3$   
push left, toward 5



$$x = 5$$

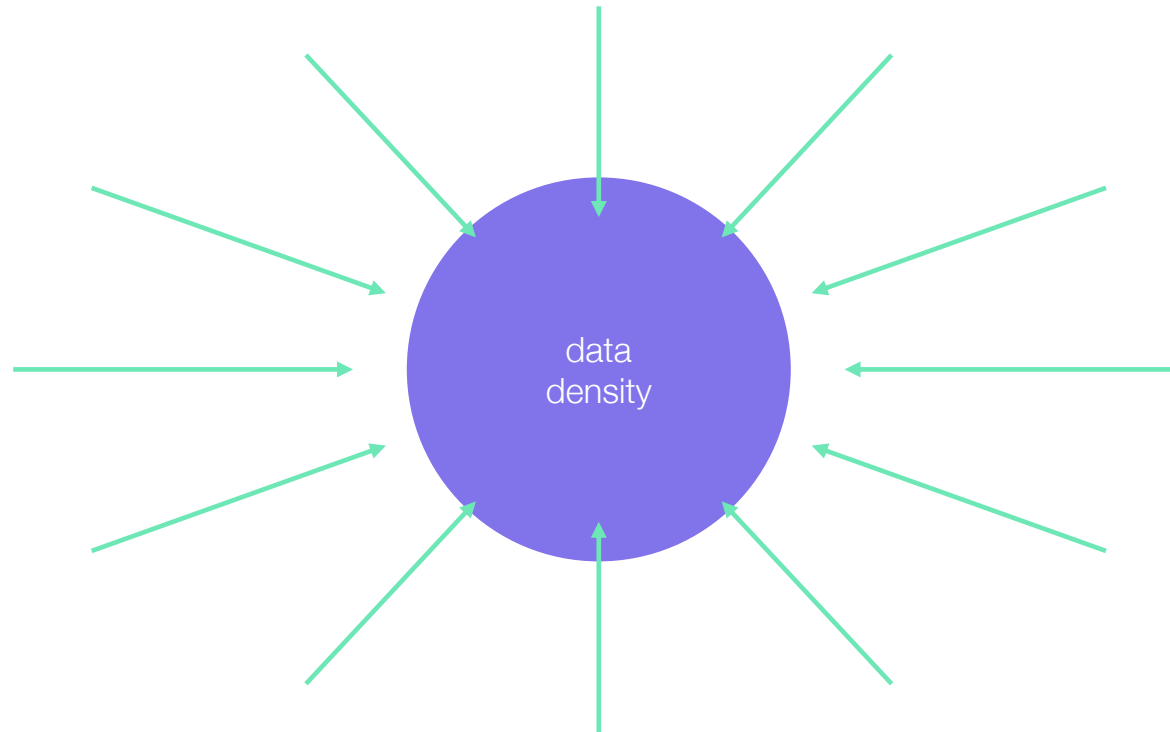
score =  $-(5-5) = 0$   
no pull — already at the peak



✨ **INSIGHT** The score is SUPPOSED to depend on  $x$  — it's the local slope AT wherever you're standing. What vanished earlier was  $Z_\theta$ , a fixed number, not  $x$ -dependence itself. Scaled to millions of pixel dimensions: the score always points from wherever you are toward wherever the data is denser.

# The Score Field: Arrows Pointing Home

Every point in space gets an arrow — its score — telling a sample which way to move to look more like real data.



Near the island the arrows are short and confident; far away (in the low-density “wastelands”) they are unreliable — which is exactly why Module 03's multi-noise-level trick is needed.

# What is “Langevin Dynamics” for Generation

- Named after Paul Langevin (1908), who wrote an equation of motion for a pollen particle jostled around in water (Brownian motion) — a classic physics problem, long before AI.
- Langevin dynamics for generation is an iterative sampling algorithm that uses the gradient of a data distribution combined with random noise to turn pure noise into realistic data samples
- Langevin's particle feels two things:
  - a drift pulling it toward a stable resting point,
  - random jitter from water molecules bumping into it.
  - We swap in our own ingredients: drift = learned score, jitter = fresh random noise.

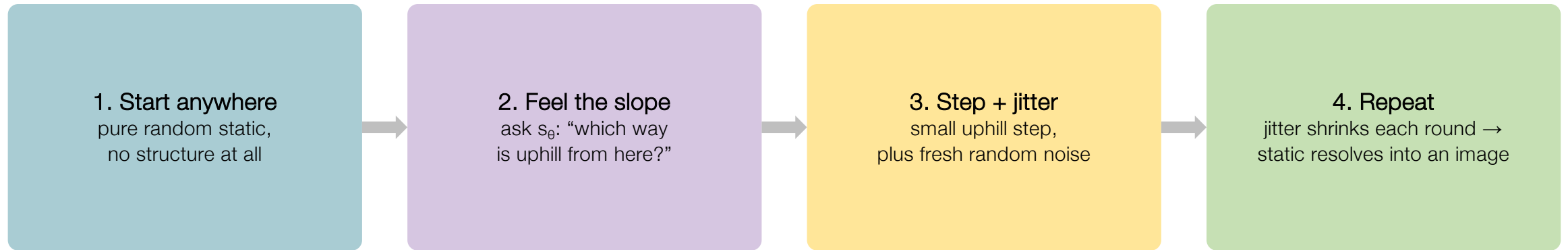
Why this is “generation,” not just optimization:

Optimization: following only the score (no jitter), every run climbs to the exact same peak — boring, not new.

Generation: The random jitter means each run settles somewhere new, anywhere the learned distribution is plausible.

So final resting point IS the generated sample.

# Sampling With Langevin Dynamics



$$\mathbf{x}_t \leftarrow \mathbf{x}_{t-1} + \text{step} \cdot \text{score} + \text{jitter} \cdot \text{noise}$$

The "I" some textbooks show ( $N(0, I)$ ) which just means:  
give every pixel its own independent random nudge — no pixel is treated specially.

# A Minimal Score-Matching Loss Function

```
energy_net = nn.Sequential(
    nn.Linear(2, hidden_units), nn.Softplus(),
    nn.Linear(hidden_units, hidden_units), nn.Softplus(),
    nn.Linear(hidden_units, 2))

logp = -energy_net(samples).sum()
grad1 = autograd.grad(logp, samples, create_graph=True)[0]
gradv = torch.sum(grad1 * vectors)
loss1 = torch.norm(grad1, dim=-1) ** 2 * 0.5

grad2 = autograd.grad(gradv, samples, create_graph=True)[0]
loss2 = torch.sum(vectors * grad2, dim=-1)
loss = (loss1 + loss2).mean() # sliced score matching loss
```

# Score-Based Models: What to Remember

- ✓ The score  $\nabla_x \log P(x)$  sidesteps the intractable normalizing constant that plain density models need.
- ✓ Scores are only reliable near real data — multi-level noise perturbation fixes this.
- ✓ Langevin dynamics turns a learned score into a sampler: repeatedly step uphill, with a little randomness.

## Score Matching Recap

Q Why does taking the gradient of  $\log P_{\theta}(x)$  remove the need to compute  $Z_{\theta}$ ?

$Z_{\theta}$  doesn't depend on  $x$ , only on  $\theta$  — so  $\nabla_x \log Z_{\theta} = 0$ . Differentiating  $\log P_{\theta}(x) = \log q(x; \theta) - \log Z_{\theta}$  with respect to  $x$  kills the second term entirely.

# Here is your problem



?



# Yeah, no one can do this



?



# Now?



?



# Now?



?



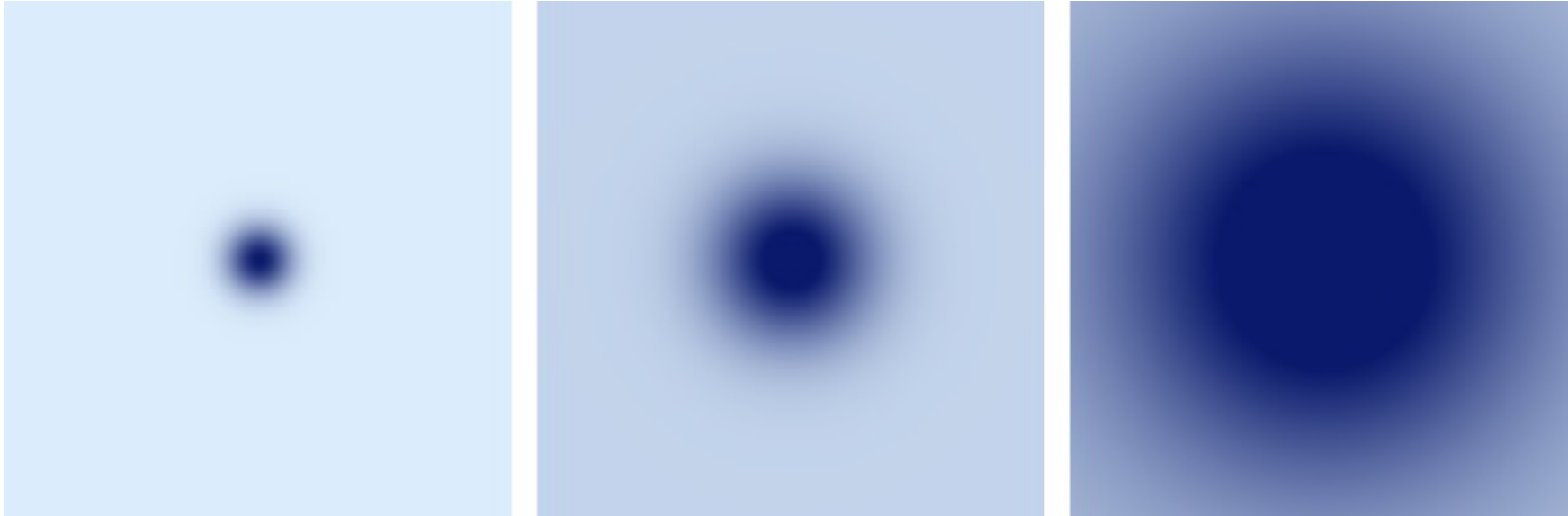
Score matching still cannot do it, since the density towards dog pictures is very low

# Denoising Diffusion Models

- 01 Recap: From Language to Pixels
- 02 Why Generating Images Is Hard
- 03 Score-Based Generative Models
- 04 Denoising Diffusion Models**
- 05 Guidance: Classifier & Classifier-Free
- 06 Latent Diffusion & Stable Diffusion
- 07 Cutting-Edge Text-to-Image: DALL·E to Today
- 08 Code Generation: Codex to Agentic Coding
- 09 Wrap-up & Takeaways

# Why Is It Called “Diffusion”?

- The name comes straight from physics — borrowed, not invented for AI.



*Drop ink in water (left) → it diffuses into a uniform haze (right)*

✨ **INSIGHT** Forward = ink diffusing into water = image dissolving into noise. Reverse = playing it backwards = noise reassembling into an image. Sohl-Dickstein et al. (2015) took this physics idea literally.

# From Noise Levels to a Diffusion Process

Sohl-Dickstein et al., 2015

Ho, Jain & Abbeel (DDPM), 2020

Song et al. (SDE), ICLR 2021

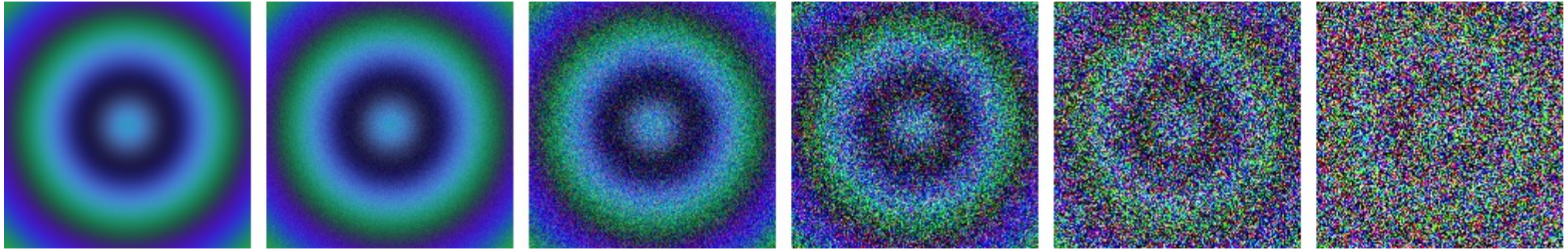
- Sohl-Dickstein et al. (2015): frame generation as reversing a gradual noising process, inspired by non-equilibrium thermodynamics.
- Ho, Jain & Abbeel (2020): [Denoising Diffusion Probabilistic Models \(DDPM\)](#) — a clean, practical training recipe that produces sharp, high-quality images.
- Song et al. (2021): unify score-based and diffusion models under one continuous-time Stochastic Differential Equation (SDE) framework.

# Forward and Reverse Processes

- Forward process: start from a real image  $x_0$  and add a little Gaussian noise at every step, until at step  $T$  it is pure noise.
- Reverse process: train a network to undo one noising step at a time, walking from pure noise back to a realistic image.
- Key insight: **predicting the noise  $\epsilon$  added at step  $t$  is mathematically equivalent to predicting the score** — diffusion models are score-based models in disguise.

$$x_t = \gamma_t \cdot x_0 + \sigma_t \cdot \epsilon, \quad \epsilon \sim N(0,1) \quad s\theta(x_t, t) = - \epsilon\theta(x_t, t) / \sigma_t$$

# Watching Diffusion Happen

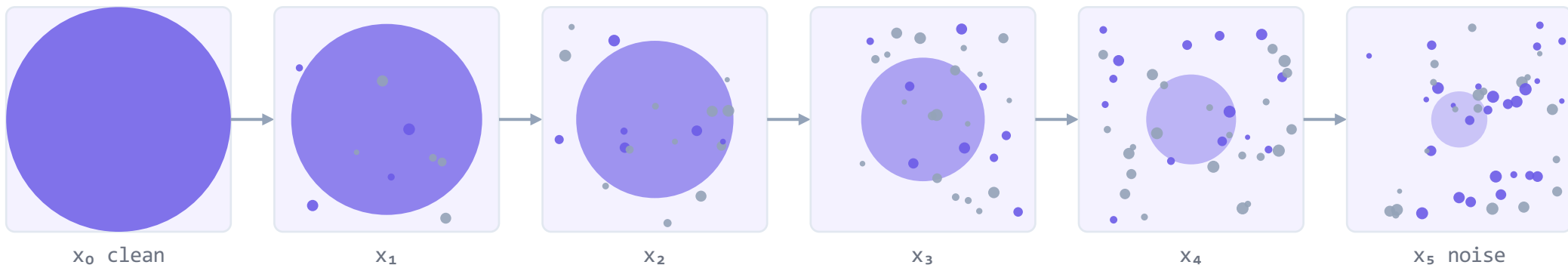


- Forward: destroy structure, one small noise step at a time (left → right above).
- Reverse: the trained network undoes this, one small step at a time — the same six snapshots, played backwards.

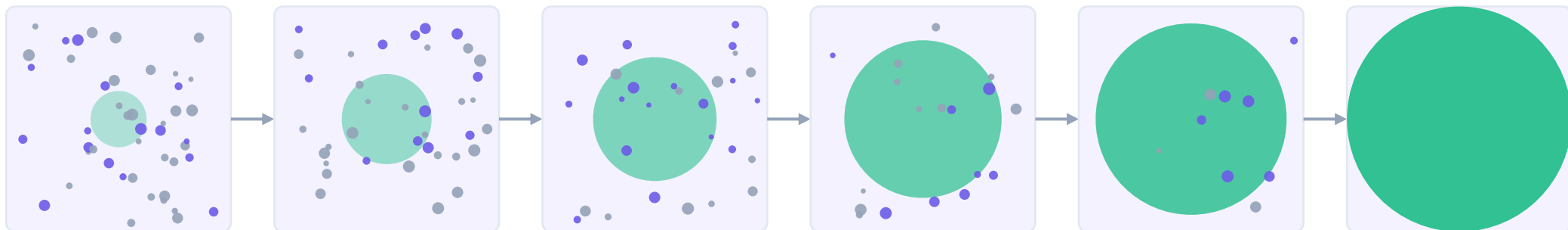
✨ **INSIGHT** At inference we only ever run the reverse direction — starting from pure noise and ending at a clean image. Every frame above is a valid image at its own noise level; there is no discontinuity, only a dial.

# Watching Diffusion Happen

Forward: destroy structure, one small noise step at a time.



Reverse: the trained network undoes this, one small step at a time.



*Same six snapshots, played backwards. At inference we only ever run the reverse direction — starting from pure noise on the right and ending at a clean image on the left.*

# Training vs. Sampling — Two Very Different Jobs

## Training

done once, ahead of time.  
Flashcard practice: “how much  
noise did I add here?”  
Millions of cards, in parallel.



## Sampling

done every time you want an image.  
Solving a jigsaw live, using what  
you learned from flashcards.  
Step by step, in order — slow.

Practicing on flashcards once (training) is what makes solving a brand-new puzzle live (sampling) possible — but the puzzle-solving itself still has to happen step by step, every single time.

# Training Objective: Just Predict the Noise

- Instead of matching a score directly, **Denoising Diffusion Probabilistic Models (DDPM)** trains a network  $\epsilon_\theta$  to predict the exact noise  $\epsilon$  mixed into  $x_t$  — turning training into simple regression.

$$\min_{\theta} \mathbb{E}_{t, x_0, \epsilon} [ \lambda(t) \cdot \| \epsilon_\theta(x_t, t) - \epsilon \|^2 ]$$

$\min_{\theta}$

adjust the network's weights  $\theta$  to make everything after this as small as possible

$\mathbb{E}_{t, x_0, \epsilon}$

average over many random choices: a timestep  $t$ , a real training image  $x_0$ , and a noise sample  $\epsilon$

$\epsilon_\theta(x_t, t)$

the network's guess: "what noise do I think is inside this image, at this timestep?"

$-\epsilon$

compare against the actual noise we injected — we know it exactly, because we added it ourselves

$\|\cdot\|^2$

squared difference between guess and truth — the everyday "how wrong was I" error

$\lambda(t)$

an optional per-timestep weight (0 to  $\infty$ ). DDPM's simplest, best-performing recipe just sets  $\lambda(t)=1$  — every timestep counts equally.

Fun fact: researchers tried "more principled" weightings like  $\lambda(t)=\sigma^2$  (perceptual) or  $\lambda(t)=\beta(t)$  (true likelihood) — but the plain, unweighted  $\lambda(t)=1$  version is what actually produces the sharpest images in practice.

# The Noise Schedule, With Real Numbers

**What is it?** A fixed recipe, decided once before training even starts, saying exactly how much noise to mix in at every timestep. It's not learned — it's a plain lookup table.

**Used twice:** (1) during training, to build each noisy “flashcard” from a clean image; (2) during sampling, so the reverse step knows exactly how big a step to undo at each  $t$ . Same schedule, both directions.

- DDPM uses a simple linear schedule for  $\beta_t$ , growing from  $\beta_1=0.0001$  to  $\beta_T=0.02$  across  $T=1000$  steps.
- From  $\beta_t$  we get how much signal survives ( $\gamma_t$ ) and how much noise has piled up ( $\sigma_t$ ) at each timestep.

| Timestep   | $\beta_t$ | $\gamma_t$ (signal) | $\sigma_t$ (noise) | What it looks like           |
|------------|-----------|---------------------|--------------------|------------------------------|
| $t = 0$    | 0.0001    | 1.00                | 0.00               | pure signal, no noise yet    |
| $t = 250$  | 0.0051    | 0.85                | 0.53               | still mostly recognisable    |
| $t = 500$  | 0.0101    | 0.55                | 0.83               | structure fading fast        |
| $t = 750$  | 0.0151    | 0.22                | 0.97               | barely any signal left       |
| $t = 1000$ | 0.0200    | 0.05                | 1.00               | indistinguishable from noise |

Notice how quickly  $\gamma_t$  (signal) collapses and  $\sigma_t$  (noise) saturates — by the halfway point, more than half the original image is already gone.

# One Reverse (Denoising) Step

Same Langevin recipe from Module 03 — but now the “score” comes from the noise-predicting network, not a direct formula.

1

## 1. Predict the noise

$\hat{\epsilon} = \epsilon\theta(x_t, t)$  — ask the network: “given this noisy image at step  $t$ , what noise do you think is baked in?”

2

## 2. Convert that into a score

$s\theta(x_t, t) = -\hat{\epsilon}/\sigma_t$  — rescale the noise guess by how much noise the schedule says should exist right now, flip its sign — now it's a direction to step, like Module 03's score.

3

## 3. Take one Langevin-style step

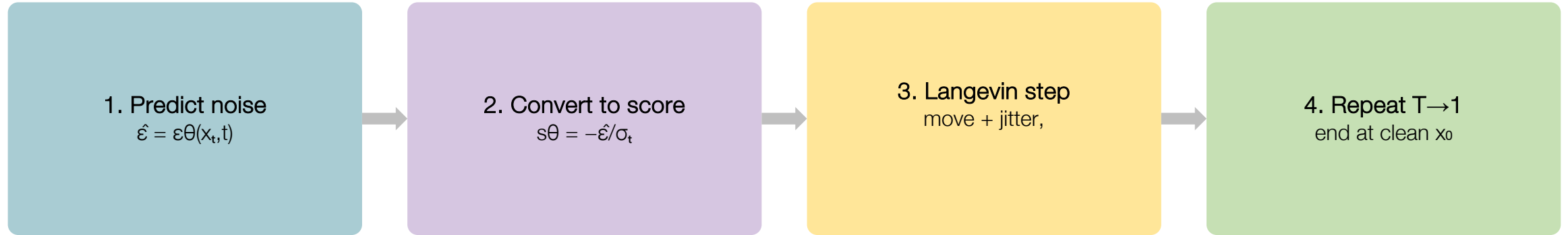
move a little toward the data (using that score), then add a small amount of fresh jitter — exactly Module 03's recipe, sized by the noise schedule (formula below).

4

## 4. Repeat

from  $t = T$  down to  $t = 1$ , ending at a clean sample  $x_0$  — this is the reverse process from Module 04's opening picture, now fully spelled out.

# One Reverse (Denoising) Step: Symbolically



$$x_{t-1} = (1/\sqrt{1-\beta_t}) \cdot (x_t + \beta_t \cdot s\theta(x_t, t)) + \sqrt{\beta_t} \cdot z$$

$1/\sqrt{1-\beta_t}$  rescales so images don't shrink/grow each step;  $\beta_t \cdot s\theta$  is the uphill nudge sized by the schedule;  $\sqrt{\beta_t} \cdot z$  is fresh random jitter, also sized by the schedule.

# Minimal DDPM Training Step (PyTorch-style)

```
def diffusion_loss(model, x0, betas, T):  
    t = torch.randint(0, T, (x0.shape[0],))  
    gamma_t = alpha_bar(betas, t).sqrt()      # signal amount  
    sigma_t = (1 - alpha_bar(betas, t)).sqrt() # noise amount  
  
    eps = torch.randn_like(x0)  
    x_t = gamma_t * x0 + sigma_t * eps         # forward process  
  
    eps_pred = model(x_t, t)                  # reverse network  
    return F.mse_loss(eps_pred, eps)          # predict the noise
```

# Diffusion Models: What to Remember

- ✓ Forward process gradually destroys structure with noise; reverse process learns to rebuild it.
- ✓ Predicting the added noise  $\epsilon$  is equivalent to predicting the score — diffusion and score-based models are the same idea.
- ✓ Training reduces to simple noise-prediction regression; sampling is what needs many iterative steps.

## One Diffusion Puzzle

Q If diffusion training is “just” MSE regression on noise, why does sampling still need dozens or hundreds of steps?

Training only teaches the network to denoise a single step at a fixed noise level. Reconstructing a full image from pure noise means repeatedly applying that single-step denoiser — an iterative reverse walk, not a one-shot jump.

# Guidance: Classifier & Classifier-Free

- 01 Recap: From Language to Pixels
- 02 Why Generating Images Is Hard
- 03 Score-Based Generative Models
- 04 Denoising Diffusion Models
- 05 Guidance: Classifier & Classifier-Free**
- 06 Latent Diffusion & Stable Diffusion
- 07 Cutting-Edge Text-to-Image: DALL·E to Today
- 08 Code Generation: Codex to Agentic Coding
- 09 Wrap-up & Takeaways

# Classifier Guidance

**Real-world motivation:** a diffusion model trained on ALL kinds of photos, asked to generate, might hand you a cat, a sunset, a car — anything plausible. You wanted specifically “golden retriever puppy.” How do we steer it?

- Bolt on a separate, already-trained classifier — the same kind of “is this a dog?” network from ordinary image classification. At each denoising step, nudge the image a little further in whatever direction makes the classifier MORE confident it's a golden retriever — on top of the normal denoising nudge.
- A scale  $\lambda$  controls how strongly the classifier's opinion pulls — too small and  $y$  is ignored, too large and image quality degrades.

$$\nabla_{\mathbf{x}} \log P(\mathbf{x}|\mathbf{y}) = \nabla_{\mathbf{x}} \log P(\mathbf{x}) + \lambda \cdot \nabla_{\mathbf{x}} \log P(\mathbf{y}|\mathbf{x})$$

Downside: this needs a bespoke classifier trained to work at every noise level of the diffusion process — expensive, and its gradient can point in unhelpful directions.

# Classifier-Free Guidance

**Real-world motivation:** text-to-image, prompt “a red bicycle.” Neat trick: skip the separate classifier entirely — train ONE diffusion model that can act both conditionally,  $p(x|y)$ , and unconditionally,  $p(x)$ .

But wait — if we “drop the label,” how do we train at all? We don't skip training on those steps. Every example still has a “prompt slot” fed into the network. ~90% of the time it holds the real caption embedding; ~10–20% of the time we swap in one fixed, special “blank” placeholder embedding instead — and train normally either way, same weights, same loss.

Like a chef handed a recipe card 90% of the time and a blank card 10% of the time: they learn to follow instructions when given a recipe, and fall back on generic cooking instinct when the card is blank — all in the same head.

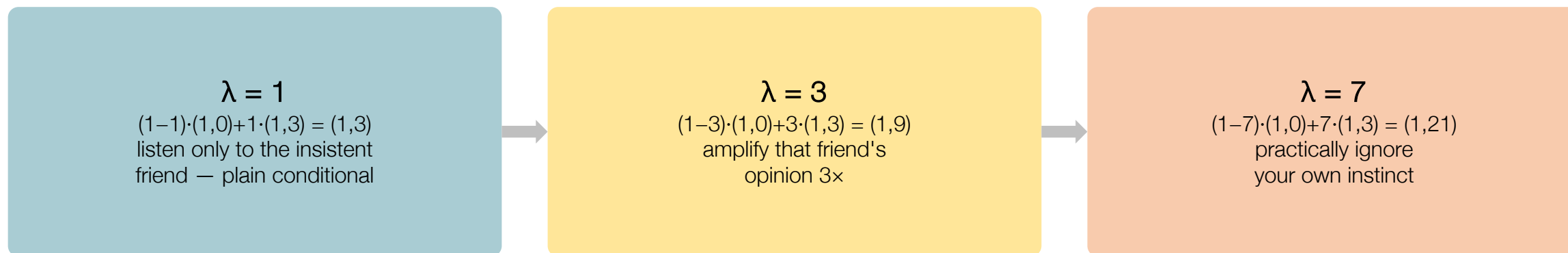
- At sampling time, run that same trained network TWICE per step — once with the real prompt embedding, once with the blank placeholder — and mix the two resulting scores.

$$\nabla_x \log P_\lambda(x|y) = (1-\lambda) \cdot \nabla_x \log P(x) + \lambda \cdot \nabla_x \log P(x|y)$$

$\lambda = 1$  recovers plain conditional generation;  $\lambda > 1$  (“guidance scale”) exaggerates the conditioning signal, trading some diversity for prompt fidelity.

# Mixing Two Scores, With Real Numbers

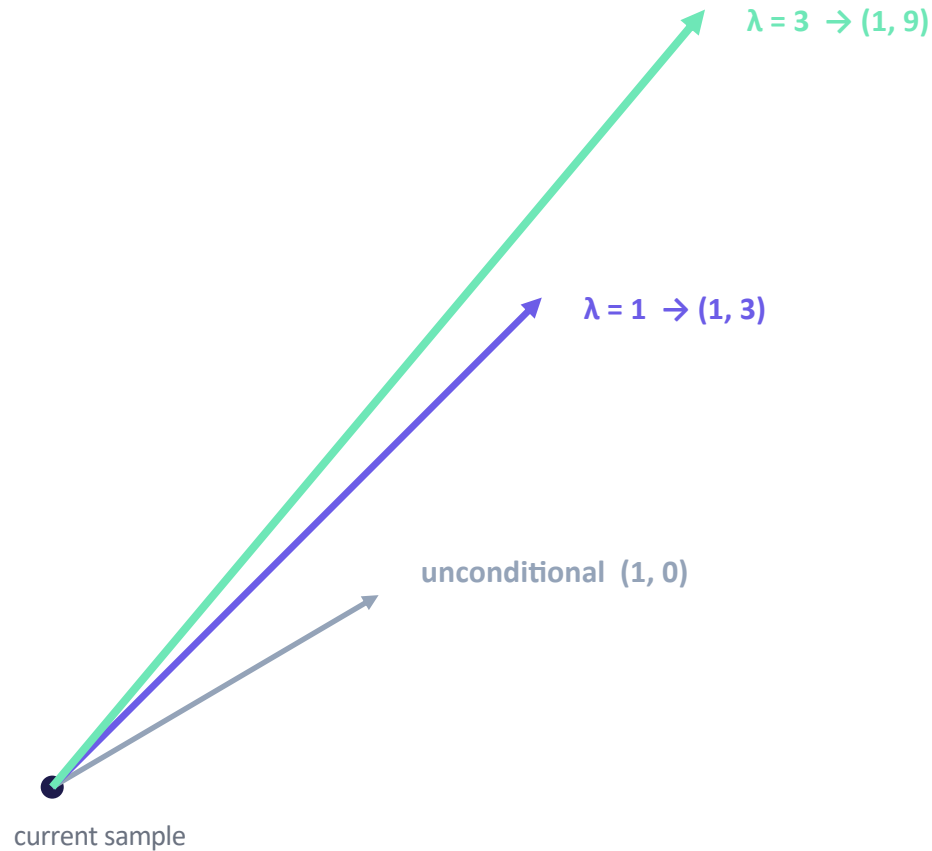
- Think of it like two friends pulling you toward a party. Your vague instinct (unconditional) nudges moderately. A friend who knows exactly which party you asked for (conditional) tugs harder, more precisely.  $\lambda$  is how much extra weight you give that insistent friend.



✨ **INSIGHT** That's the guidance scale in one sentence: it doesn't invent a new direction, it just turns up the volume on the prompt's own opinion — the same trick from the panda example, expressed as arithmetic you can check by hand.

# Mixing Two Scores, With Real Numbers: Visualization

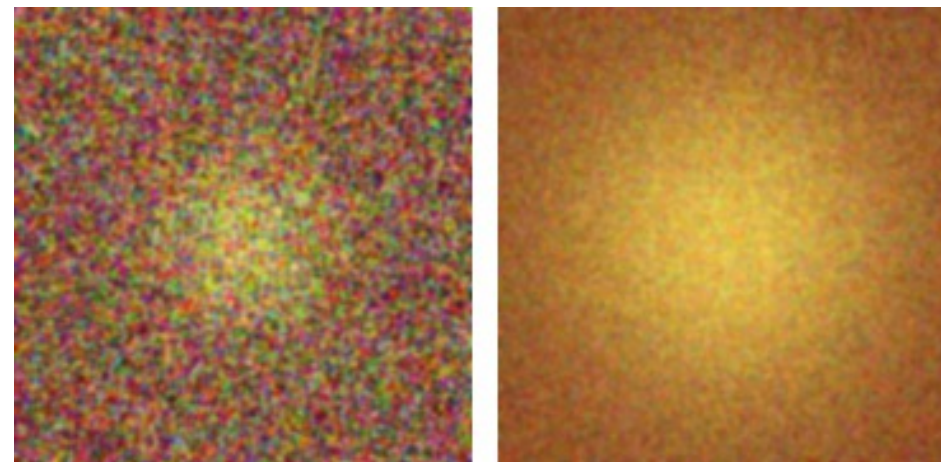
- Think of it like two friends pulling you toward a party. Your vague instinct (unconditional) nudges moderately. A friend who knows exactly which party you asked for (conditional) tugs harder, more precisely.  $\lambda$  is how much extra weight you give that insistent friend.



# Guidance Scale in Practice

Nichol et al., GLIDE, 2021

- OpenAI's GLIDE model, prompted with “A stained glass window of a panda eating bamboo.”
- At guidance scale 1 (no extra guidance), samples are diverse but only loosely follow the prompt — some barely resemble a panda or stained glass at all.
- At guidance scale 3, samples much more consistently show a clear panda in stained-glass style — at some cost to sample diversity.



*illustrative sketch, not actual GLIDE output — left: scale 1, right: scale 3*

✨ **INSIGHT** Rule of thumb:  $\uparrow$  guidance scale  $\Rightarrow$   $\uparrow$  prompt fidelity,  $\downarrow$  diversity — push far enough and you start seeing oversaturated, cartoonish artifacts as the model over-commits to the prompt.

## Guidance: What to Remember

- ✓ Classifier guidance steers sampling with a separate, noise-aware classifier and Bayes' rule.
- ✓ Classifier-free guidance folds conditioning into one model via label dropout — no extra classifier needed.
- ✓ The guidance scale  $\lambda$  is a fidelity ↔ diversity dial, tuned per application.

**Q** Why can classifier-free guidance use the same network for both  $p(x)$  and  $p(x|y)$ ?

Because the label is just an extra input the model conditions on; replacing it with a fixed “no label” token during training teaches the same network to also behave unconditionally.

# Latent Diffusion & Stable Diffusion

- 01 Recap: From Language to Pixels
- 02 Why Generating Images Is Hard
- 03 Score-Based Generative Models
- 04 Denoising Diffusion Models
- 05 Guidance: Classifier & Classifier-Free
- 06 Latent Diffusion & Stable Diffusion**
- 07 Cutting-Edge Text-to-Image: DALL·E to Today
- 08 Code Generation: Codex to Agentic Coding
- 09 Wrap-up & Takeaways

# Why Run Diffusion in Pixel Space at All?

Rombach et al., CVPR 2022

- Diffusion in full pixel space (e.g.  $512 \times 512 \times 3$ ) is accurate but very expensive — every training and sampling step touches millions of numbers.
- Rombach et al. (2022), “High-Resolution Image Synthesis with Latent Diffusion Models”: [compress images into a small latent space first](#), then run the entire diffusion process there.
- Result: Stable Diffusion — the same score/diffusion machinery from Modules 3–4, made practical on consumer GPUs.

# Three Pieces Working Together

1

## VAE Encoder / Decoder

Compresses a pixel image  $x$  into a small latent  $z$  (a compressed, learned summary — not a picture itself), and decodes a denoised latent back into pixels.

2

## Denoising U-Net

The same  $\epsilon_{\theta}(z_t, t)$  noise-prediction network from Module 04. In one line: it shrinks the latent down (to see the big picture), then grows it back up (to fill in detail), with shortcuts between matching sizes so detail isn't lost.

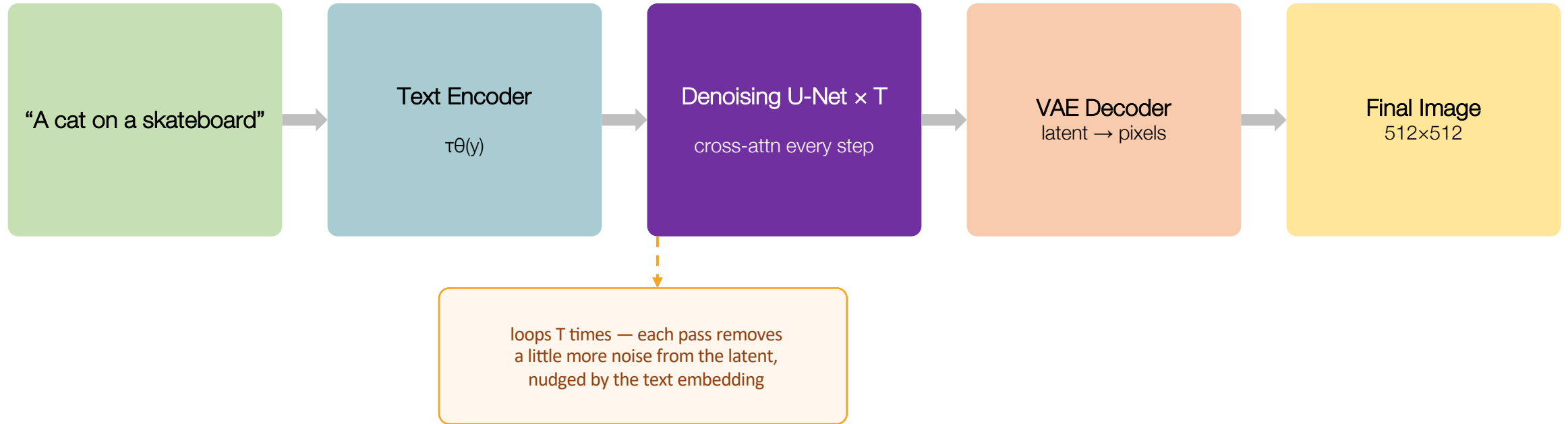
3

## Text Encoder + Cross-Attention

Turns a prompt into a sequence of embeddings  $\tau_{\theta}(y)$ , injected into the U-Net so the denoising is guided by the text.

Pipeline: Text  $\rightarrow$  Text Encoder  $\rightarrow$  [Cross-Attention into U-Net]  Noisy Latent  $\rightarrow$  Denoising U-Net  $\times T$  steps  $\rightarrow$  Clean Latent  $\rightarrow$  VAE Decoder  $\rightarrow$  Image.

# The Full Journey: Prompt to Pixels



The U-Net box is really a loop — the same denoising step from Module 4, repeated T times, checking in with the text prompt at every step via cross-attention.

# Why Latent Space Is $\approx 48\times$ Cheaper

- A 512×512 RGB image has  $512 \times 512 \times 3 = 786,432$  numbers for the U-Net to denoise at every single step.
- Stable Diffusion's VAE compresses this by 8× in each spatial dimension, to a 64×64 latent grid — but with 4 channels instead of 3.
  - Quick recap — “channels”: in a normal photo, 3 channels are the separate Red/Green/Blue grids stacked together. A latent's 4 channels are the same idea — 4 stacked grids of numbers — except each one is a learned, abstract feature rather than a literal colour.

Pixel space:  $512 \times 512 \times 3 = 786,432$  numbers

Latent space:  $64 \times 64 \times 4 = 16,384$  numbers

Compression:  $786,432 \div 16,384 \approx 48\times$  fewer numbers per denoising step

That  $\sim 48\times$  shrinkage, repeated across every one of the T denoising steps, is what turns “requires a data-centre” into “runs on a gaming laptop.”

✨ **INSIGHT** That  $\sim 48\times$  shrinkage, repeated across every one of the T denoising steps, is what turns “requires a data-centre” into “runs on a gaming laptop.”

# Text Conditioning via Cross-Attention

Quick recap of three jargon words this slide leans on:

## Noisy latent

The small, compressed image-in-progress (Module 06's  $z_t$ ) with some of Module 04's noise still mixed in — the canvas the U-Net is currently working on.

## U-Net

The painter: shrinks the canvas down to see the big picture, then builds it back up to add detail, one denoising step at a time.

## Cross-attention

The painter glancing at a written description while painting: “what do the words say should be here?” — same Q/K/V idea as text self-attention, just across two modalities.

- In our painter analogy: the Query is “what patch of canvas am I working on right now?” (comes from the noisy latent). The Key and Value are “what do the words say to look for / paint?” (both come from the text embedding).

$$\text{Attention}(Q,K,V) = \text{softmax}(QK^T/\sqrt{d}) \cdot V, \quad Q=W_{uu} \cdot \phi(z_t), \quad K=W_k \cdot \tau\theta(y), \quad V=W_v \cdot \tau\theta(y)$$

- The overall training loss looks just like the DDPM loss from Module 04, with the text embedding  $\tau\theta(y)$  added as extra conditioning — same “predict the noise” game, just with the words available as a hint.

$$L_{\text{LDM}} = \mathbb{E}[\| \epsilon - \epsilon\theta(z_t, t, \tau\theta(y)) \|^2]$$

# “Stable Diffusion Is Unstable” (2024)

Du, Li, Qiu & Xu, University of Sydney, 2024

- Observation: when a prompt names an object A, adding or swapping in a second noun B often makes A vanish from the image entirely — replaced by B, not shown alongside it.
- Example: “a street sign that reads ‘Latent Diffusion’” sometimes renders garbled text once other prompt elements compete for the model's attention.

Reminder: these are powerful, but still statistical pattern-matchers — cross-attention conditioning can be pulled unevenly between competing concepts in a prompt.

# Latent Diffusion: What to Remember

- ✓ Running diffusion in a compressed latent space (not raw pixels) is what makes Stable Diffusion fast enough to use.
- ✓ Cross-attention is the bridge between text embeddings and the image-denoising U-Net — same Q/K/V idea as text self-attention.
- ✓ The training loss is still “predict the noise,” just now conditioned on a text embedding.

# Latent Diffusion Trivia

**Q** In the cross-attention layer, which of Q, K, V come from the image latent, and which from the text prompt?

Q comes from the noisy image latent  $z_t$  — “what am I currently rendering?” K and V both come from the text embedding  $\tau\theta(y)$  — “what does the prompt say to look for / paint?”

# Cutting-Edge Text-to-Image: DALL·E to Today

- 01 Recap: From Language to Pixels
- 02 Why Generating Images Is Hard
- 03 Score-Based Generative Models
- 04 Denoising Diffusion Models
- 05 Guidance: Classifier & Classifier-Free
- 06 Latent Diffusion & Stable Diffusion
- 07 Cutting-Edge Text-to-Image: DALL·E to Today**
- 08 Code Generation: Codex to Agentic Coding
- 09 Wrap-up & Takeaways

# DALL·E → DALL·E 2: Where CLIP Enters the Story

Ramesh et al., DALL·E, 2021

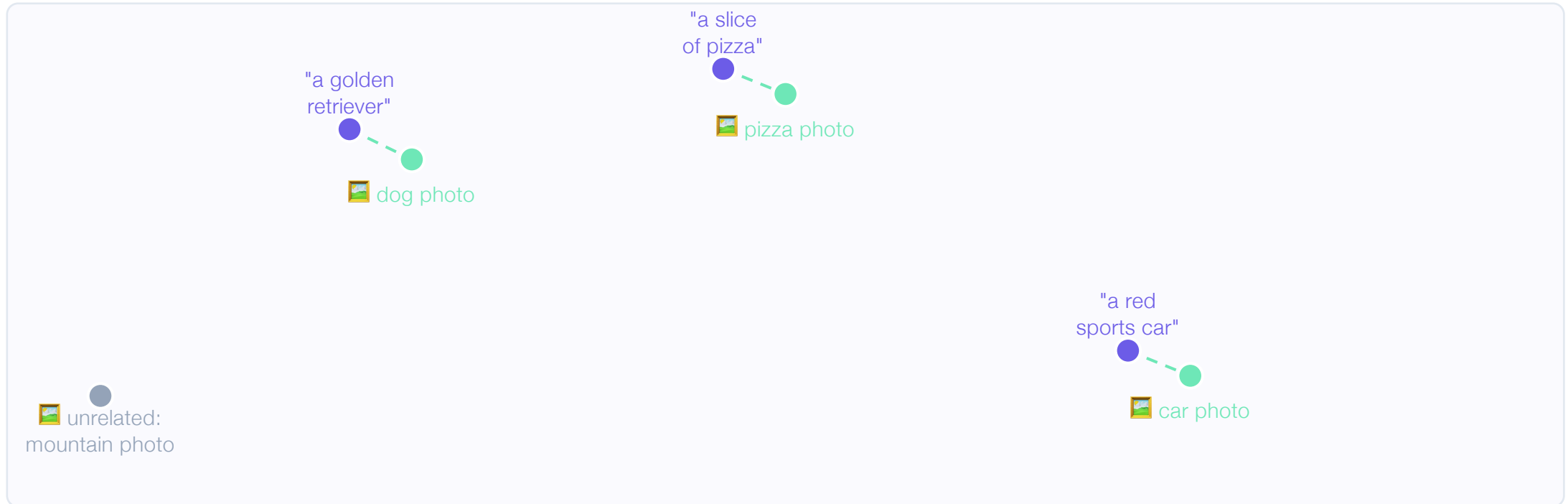
Radford et al., CLIP, 2021

Ramesh et al., DALL·E 2, 2022

- DALL·E (2021): treats text-to-image as sequence generation — a discrete VAE tokenizes images into a grid of “image tokens,” and an autoregressive Transformer predicts text tokens then image tokens, just like Module 4's decoder-only recipe.
- CLIP (2021): trains a text encoder and an image encoder jointly so matching text/image pairs land close together in one shared embedding space.
- DALL·E 2 (2022): replaces the autoregressive image decoder with a [diffusion model guided by CLIP embeddings](#) — combining Modules 3–6 with CLIP's shared representation space.

# CLIP's Shared Embedding Space

CLIP trains text and image encoders together so that a caption and its matching photo land in nearly the same spot — even though one started as words and the other as pixels.



Matching pairs (dashed lines) sit close together; the unrelated mountain photo sits far from every caption. That closeness is exactly the signal DALL·E 2's diffusion decoder is conditioned on.

## How Does CLIP Actually Learn This?

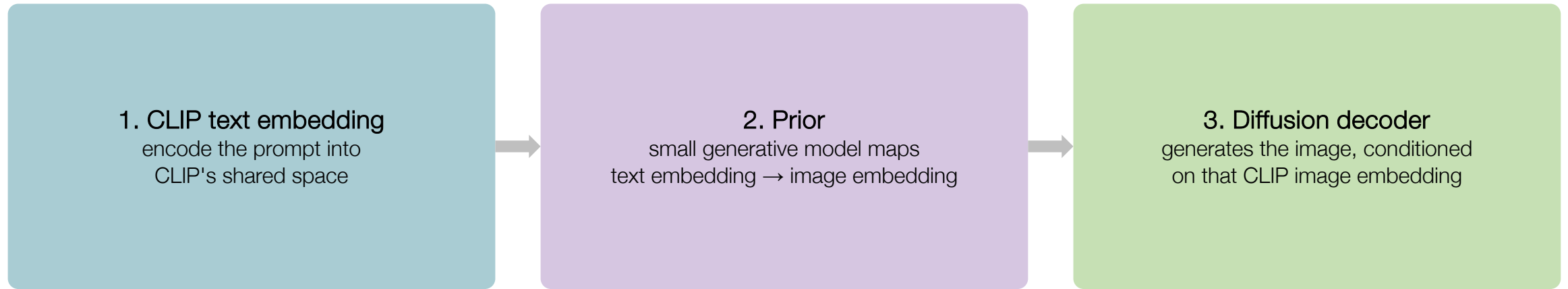
- The training trick is called contrastive learning: pull matching pairs together, push every mismatched pair apart — all at once, in every batch.

Take a batch of 3 (caption, photo) pairs. Encode all captions and all photos, then score every caption against every photo:

|         |  photo |  photo |  photo |
|---------|-------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| "a dog" | pull together                                                                             | push apart                                                                                | push apart                                                                                |
| "a car" | push apart                                                                                | pull together                                                                             | push apart                                                                                |
| "pizza" | push apart                                                                                | push apart                                                                                | pull together                                                                             |

✨ **INSIGHT** Only the diagonal (3 correct pairs) gets pulled close; all 6 off-diagonal combinations in this one 3×3 batch get pushed apart simultaneously. Repeat over millions of real (caption, photo) pairs scraped from the web, and the shared space from the previous slide falls out for free — no manual labels beyond “these two came together.”

# Prior + Decoder: How DALL·E 2 Uses CLIP



Same recipe as Stable Diffusion underneath — the novelty is using CLIP's joint embedding space as the conditioning signal instead of a raw text encoder.

# 2023–25: Better Prompt Following — DALL·E 3 & SDXL

Betker et al., DALL·E 3, OpenAI 2023

Podell et al., SDXL, 2023

- Most web image captions are short and vague (“dog”) — so the model never learned that fine detail follows fine language.
- Fix: re-caption the entire training set with a dedicated image-captioning model, producing long, highly descriptive synthetic captions before diffusion training even starts.
- SDXL: a larger UNet (2.6B parameters), two text encoders concatenated for richer conditioning, plus a second “refiner” diffusion model that polishes high-frequency detail in the final denoising steps.

✨ **INSIGHT** DALL·E 3's core insight: the training data was the bottleneck, not the diffusion model. Better pixels didn't come from a bigger denoiser — they came from finally telling it, in words, what it was looking at.

# 2024: Rethinking the Objective — Flow Matching & SD3

Esser et al., Stable Diffusion 3, ICML 2024

Lipman et al., Flow Matching, 2023

- DDPM's noise schedule (Module 4) is one specific path from data to noise. Flow matching / rectified flow generalizes this: learn a velocity field along the STRAIGHTEST possible path between the noise distribution and the data distribution.
- Straighter paths → **fewer sampling steps needed** for the same quality — the reverse walk from Module 4 has less zig-zagging to do.
- SD3 also swaps the U-Net for a Multimodal Diffusion Transformer (MM-DiT): text and image tokens flow through the SAME Transformer blocks with joint attention, rather than text only entering via cross-attention.

$v_\theta(x_t, t) \approx dx_t/dt$  — predict the velocity along a straight path, not just the noise

# Making Diffusion Fast: Consistency Models

Song et al., Consistency Models, ICML 2023

Luo et al., LCM, 2023

- The real cost of diffusion isn't training — it's sampling: dozens to hundreds of sequential steps, every single time (Module 4's training-vs-sampling asymmetry).
- Consistency models train a network to jump **directly from ANY point on the noising trajectory straight back to the clean image**, in one shot — by forcing nearby points on the same trajectory to map to the same output (“self-consistency”).
- Latent Consistency Models (LCM) distill an already-trained Stable Diffusion model this way — turning a 50-step sampler into a 1–4 step sampler, with only a modest quality trade-off.

✨ **INSIGHT** Worked intuition: instead of Module 4's slow “walk downhill one small step at a time,” a consistency model learns to predict “where does this entire downhill walk end up?” directly — trading a little accuracy for a 10–50× speedup.

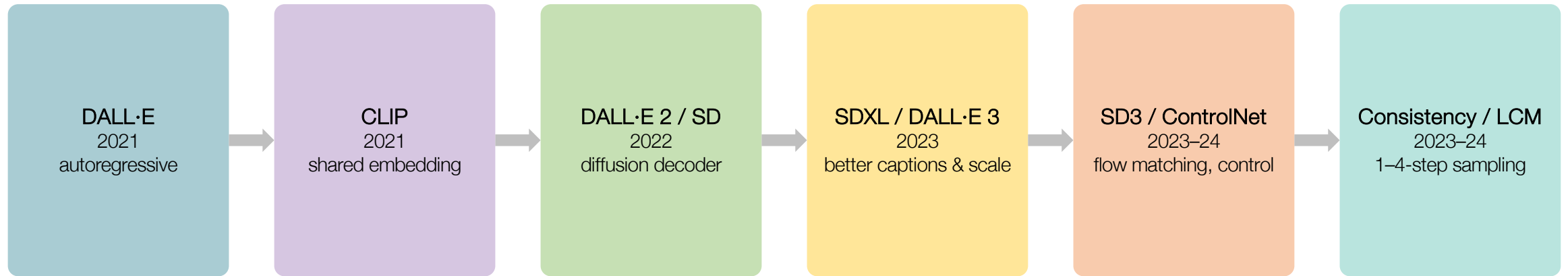
# Steering Diffusion Spatially: ControlNet

Zhang, Rao & Agrawala, ControlNet, ICCV 2023

- Text prompts alone can't say “put the arm exactly here” or “match this exact building outline.” ControlNet adds a second, spatial conditioning signal on top of text: an edge map, a human pose skeleton, a depth map, a scribble.
- Trick: clone the pretrained U-Net's encoder into a trainable copy, zero-initialize its output layers, and add its output back into the frozen original network — so training starts as a perfect no-op and only gradually learns to inject the new spatial signal.

✨ **INSIGHT** Zero-initialization is the trick worth sitting with: at step 0, ControlNet's contribution is exactly zero, so training starts from a perfectly working model and only gradually learns to add spatial control — no risk of breaking what already works.

# The Text-to-Image Timeline, Updated



The underlying machinery from Modules 3–6 hasn't changed — every advance since is better data, a straighter training objective, a smarter distillation trick, or an extra conditioning channel.

## DALL·E & Friends Trivia

**Q** A friend says “consistency models must be a totally different kind of generative model from diffusion.” Are they right?

Not quite — a consistency model is typically distilled FROM an already-trained diffusion model. It reuses the same forward noising trajectory (Module 4); it just learns a shortcut that jumps from anywhere on that trajectory straight to the clean endpoint, instead of walking it step by step.

# Code Generation: Codex to Agentic Coding

- 01 Recap: From Language to Pixels
- 02 Why Generating Images Is Hard
- 03 Score-Based Generative Models
- 04 Denoising Diffusion Models
- 05 Guidance: Classifier & Classifier-Free
- 06 Latent Diffusion & Stable Diffusion
- 07 Cutting-Edge Text-to-Image: DALL·E to Today
- 08 Code Generation: Codex to Agentic Coding**
- 09 Wrap-up & Takeaways

# What Came Before: Autocomplete, Not Understanding

- The core problem, from the start: writing code requires understanding intent — but early tools only ever matched patterns they'd literally seen before.



# Code Is Just Another Language

Chen et al., Codex, 2021

Li et al., StarCoder, 2023

- Codex's insight (2021): stop treating code as a special case — reuse the exact decoder-only Transformer recipe that already works for English (Modules 0–1).
- Code has just as much statistical structure as prose — arguably more, thanks to rigid syntax — so “predict the next token” transfers directly.
- Codex (2021): GPT-3 further trained on public source code; powers the original GitHub Copilot. [StarCoder \(2023\): fully open, 15.5B-parameter decoder-only Transformer, ~1 trillion tokens of 80+ languages.](#)

Same architecture, same training objective as text — the leap forward is entirely about data (billions of lines of real code) and scale, not a new mechanism.

# Left-to-Right Isn't How Editing Works

- Plain next-token training only ever conditions on what comes before the cursor — fine for writing a fresh document top to bottom.
- But real code editing happens with the cursor in the middle of an existing file — there's already code both above AND below the gap you're filling.
- A left-to-right-only model is completely blind to that “**suffix**” — it has never once seen what comes after a gap during training.

Everyday example: your cursor sits inside a function whose signature and return statement already exist — you need to fill the body in between. A model that can only “continue forward” has no way to use the return statement as a hint.

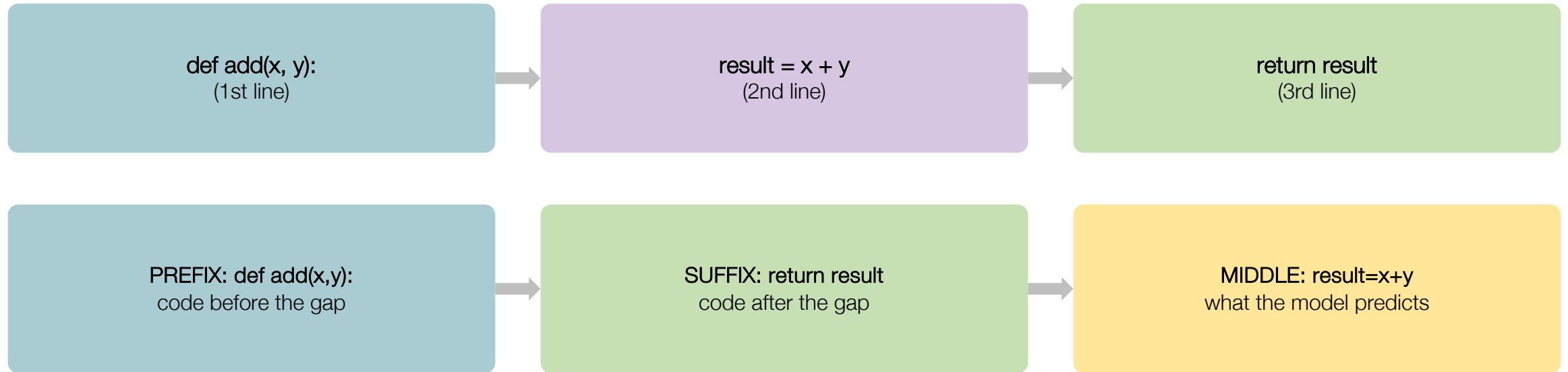
# Repo-Level Context & Fill-in-the-Middle (FIM)

- Repo-as-document: training examples are prefixed with special tokens like <reponame> and <filename>, so the model learns cross-file context the way it would learn document structure.
- Fill-in-the-Middle: reorder training data into **prefix, then suffix, then middle**. The model still only ever predicts left-to-right — we just feed it the surrounding code in a cleverer order first.

```
<fim_prefix> def add(x, y):  
<fim_suffix>     return result  
<fim_middle>     result = x + y
```

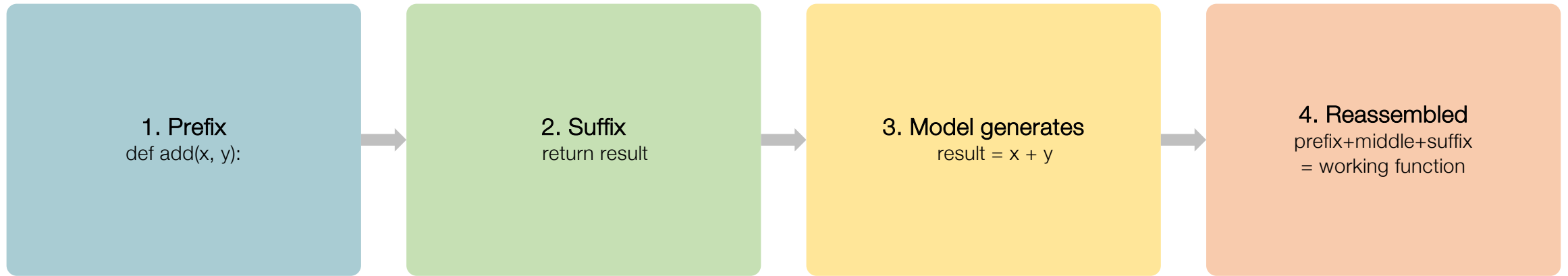
# Shuffling the Deck: Why FIM Works

- Natural reading order (top) vs. how FIM rearranges the very same three chunks for training (bottom):



✨ **INSIGHT** The model still only ever predicts left-to-right — the trick is purely in the order we FEED it the file. That single reshuffle is enough to make it condition on code that, chronologically, doesn't exist yet.

# Fill-in-the-Middle, Step by Step



# A Trickier Fill-in-the-Middle

- Here the missing piece must reuse a variable from the prefix and match the suffix's return line — not just “continue” the code.

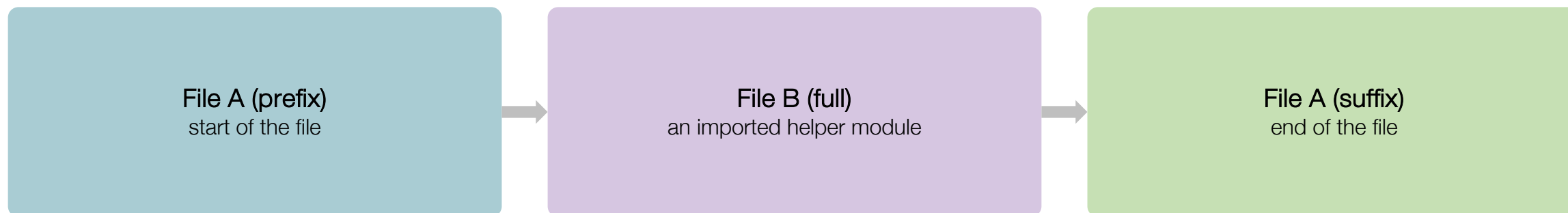
```
Prefix:  def total_of_evens(nums):  
         total = 0  
         for n in nums:  
Suffix:      return total
```

```
Model must infer (uses both sides):  
    if n % 2 == 0:  
        total += n
```

✨ **INSIGHT** The model had to reuse the exact variable name `total` from the prefix, at the exact indentation the suffix expects — only possible because FIM training showed it both sides at once.

# FIM at Scale: Combining Whole Repos

- The same trick works across MULTIPLE files. Imagine handing someone the first half of file A, the ENTIRE helper file B that A imports from, then the ending of file A — and asking them to fill in A's missing middle, using B's signatures as a hint.



Repo files are concatenated (with <filename> separators) up to the model's context window (e.g. 8K tokens for StarCoder) — then FIM's prefix/suffix/middle split is applied across that WHOLE packed sequence. Training itself doesn't change: still the same next-token loss from Module 4. All the cleverness is in how the input document is assembled.

# What a StarCoder Training Example Looks Like

```
<reponame>huggingface/transformers<filename>src/transformers/trainer.py<gh_stars>...
```

```
# This function initializes the training loop
```

```
def train(model, dataloader, optimizer, epochs):
```

```
    for epoch in range(epochs):
```

```
        for batch in dataloader:
```

```
            optimizer.zero_grad()
```

```
            output = model(batch)
```

```
            loss = output.loss
```

```
            loss.backward()
```

```
            optimizer.step()
```

```
<|endoftext|>
```

# From StarCoder to Today's Code Models

Rozière et al., Code Llama, 2023

Lozhkov et al., StarCoder2, 2024

| Model                          | Year  | What changed                                                                                                        |
|--------------------------------|-------|---------------------------------------------------------------------------------------------------------------------|
| Codex                          | 2021  | GPT-3 fine-tuned on code; powered the first Copilot                                                                 |
| StarCoder                      | 2023  | fully open weights, repo-context + FIM, 15.5B params                                                                |
| Code Llama                     | 2023  | Llama 2 further trained on code; long-context (100K) variants; a dedicated “Instruct” chat variant                  |
| StarCoder2                     | 2024  | trained with grouped-query + sliding-window attention for longer, cheaper context; broader repo-level data curation |
| Frontier code models (2024–26) | today | same decoder-only + FIM foundations, now wrapped in agentic tool-use loops (next slides)                            |

# Measuring Code Models: pass@k

Chen et al., HumanEval, 2021

Austin et al., MBPP, 2021

- Unlike text, generated code can be checked automatically: does it pass a set of unit tests? HumanEval (164 hand-written problems) and MBPP are the classic benchmarks.

$$\text{pass@k} = \mathbb{E} [ 1 - C(n-c, k) / C(n, k) ]$$

- $n$  = samples generated per problem,  $c$  = how many pass all tests,  $k$  = attempts we'd actually submit

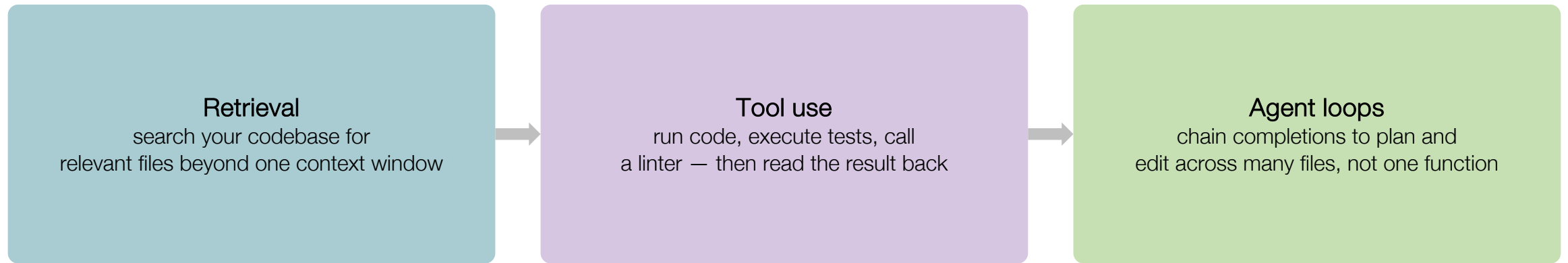


*$n = 10$  samples generated for one problem — 3 pass all unit tests, 7 fail*

✨ **INSIGHT**  $\text{pass@1} \approx c/n = 30\%$  (one random attempt).  $\text{pass@10} = 100\%$  — across all 10 attempts, at least one passes, since  $c \geq 1$ . This is why practical tools let the model try, run tests, and retry — turning  $\text{pass@k}$  into an achievable target rather than a one-shot bet.

# From Autocomplete to Agentic Coding

- The decoder-only Transformer we studied is just the engine. Modern coding assistants (Copilot, Cursor, Claude Code, and others) wrap it with several extra tools:



The loop: plan → edit → run tests → read the failure → fix → repeat. This closes exactly the gap pass@k exposed — the model gets to actually check its own work, the way a developer would, instead of guessing blind in one shot.

# Benchmarking the Agent, Not Just the Snippet

Jimenez et al., SWE-bench, ICLR 2024

- HumanEval asks for one self-contained function. Real work is closer to: “here's a GitHub issue on a 50,000-line repo — fix it.”
- SWE-bench draws its problems from real GitHub issues and their real merged fixes, scored by whether the model's patch makes the repo's own hidden test suite pass.
- This forces exactly the skills Module 8 has built up to: reading repo-level context (FIM's prefix/suffix idea, scaled to a whole codebase), retrieving the right files, and iterating with tool feedback — not just completing one function in isolation.

✨ **INSIGHT** Not a synthetic toy problem: every task is a real bug a real developer actually fixed, in a real production repo. If a model solves it, that solution has to survive the same messy conditions the human's did.

# Open Challenges, Still Unsolved

| Challenge                  | What goes wrong                                                                                                  |
|----------------------------|------------------------------------------------------------------------------------------------------------------|
| Hallucinated APIs          | confidently inventing functions or parameters that don't exist                                                   |
| Context limits             | huge codebases still don't fully fit — the “left-to-right can't see everything” problem re-appears at repo scale |
| Fast-moving libraries      | training data goes stale as real-world APIs change                                                               |
| Correctness $\neq$ fluency | fluent-looking code isn't the same as correct or safe code                                                       |
| Evaluation itself          | pass@k and SWE-bench are progress, but still narrower than “is this good, maintainable software?”                |

# Code Generation: What to Remember

- ✓ Codex, StarCoder, Code Llama and today's agents all reuse the decoder-only Transformer recipe from Modules 0–1 — the gains come from data, objective design, and tool-use loops.
- ✓ Fill-in-the-Middle training matches how developers actually edit code: filling a gap, not just continuing to the right — scaled from one function to whole repos.
- ✓ pass@k and SWE-bench moved evaluation from “does it look right” to “does it run and fix the real issue” — but hallucination, context limits and correctness remain open.

# One Last Puzzle

**Q** Why would a plain left-to-right model struggle at “complete this function body given the signature above and the return statement below”?

Ordinary next-token training only ever conditions on what comes before the cursor — it has never seen the suffix during training, so it can't use it. FIM training explicitly reorders prefix/suffix/middle so the model learns to condition on both sides of a gap.

# Wrap-up & Takeaways

- 01 Recap: From Language to Pixels
- 02 Why Generating Images Is Hard
- 03 Score-Based Generative Models
- 04 Denoising Diffusion Models
- 05 Guidance: Classifier & Classifier-Free
- 06 Latent Diffusion & Stable Diffusion
- 07 Cutting-Edge Text-to-Image: DALL·E to Today
- 08 Code Generation: Codex to Agentic Coding
- 09 **Wrap-up & Takeaways**

# Key Takeaways

| Topic                     | One-line takeaway                                                                                                                       |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Score Matching            | Learn $\nabla_x \log P(x)$ instead of $P(x)$ itself — sidesteps the intractable normalizing constant.                                   |
| Diffusion Models          | Predicting added noise $\varepsilon \approx$ predicting the score; sampling reverses a gradual noising process.                         |
| Guidance                  | Classifier-free guidance mixes conditional and unconditional scores to control prompt fidelity.                                         |
| Latent Diffusion / DALL·E | Compress, condition (cross-attention or CLIP), or repurpose the same Transformer for a new modality.                                    |
| Cutting-edge T2I          | Better captions (DALL·E 3), straighter paths (flow matching / SD3), faster sampling (consistency models), spatial control (ControlNet). |
| Code Generation           | Repo context + Fill-in-the-Middle turn the text recipe into a code recipe; agent loops + tool use close the loop pass@k exposed.        |

# References

1. Hyvärinen, A., Estimation of non-normalized statistical models by score matching, JMLR 6, 2005.
2. Song, Y. & Ermon, S., Generative Modeling by Estimating Gradients of the Data Distribution, NeurIPS 2019.
3. Ho, J., Jain, A. & Abbeel, P., Denoising Diffusion Probabilistic Models, arXiv:2006.11239, 2020.
4. Song, Y. et al., Score-Based Generative Modeling through Stochastic Differential Equations, ICLR 2021.
5. Nichol, A. et al., GLIDE: Towards Photorealistic Image Generation and Editing with Text-Guided Diffusion Models, 2021.
6. Radford, A. et al., Learning Transferable Visual Models From Natural Language Supervision (CLIP), ICML 2021.
7. Ramesh, A. et al., Hierarchical Text-Conditional Image Generation with CLIP Latents (DALL·E 2), 2022.
8. Rombach, R. et al., High-Resolution Image Synthesis with Latent Diffusion Models, CVPR 2022.
9. Chen, M. et al., Evaluating Large Language Models Trained on Code (Codex / HumanEval), 2021.
10. Li, R. et al., StarCoder: May the Source Be With You!, arXiv:2305.06161, 2023.
11. Betker, J. et al., Improving Image Generation with Better Captions (DALL·E 3), OpenAI, 2023.
12. Podell, D. et al., SDXL: Improving Latent Diffusion Models for High-Resolution Image Synthesis, 2023.
13. Song, Y. et al., Consistency Models, ICML 2023. Luo, S. et al., Latent Consistency Models, 2023.
14. Zhang, L., Rao, A. & Agrawala, M., Adding Conditional Control to Text-to-Image Diffusion Models (ControlNet), ICCV 2023.
15. Rozière, B. et al., Code Llama: Open Foundation Models for Code, 2023.
16. Esser, P. et al., Scaling Rectified Flow Transformers for High-Resolution Image Synthesis (SD3), ICML 2024.
17. Lozhkov, A. et al., StarCoder 2 and The Stack v2, 2024.
18. Jimenez, C. et al., SWE-bench: Can Language Models Resolve Real-World GitHub Issues?, ICLR 2024.
19. Du, C., Li, Y., Qiu, Z. & Xu, C., Stable Diffusion Is Unstable, University of Sydney, 2024